

performance on real world inputs. Before we discuss why NLP models don't generalise well across data sets, we need to understand the differences between extrapolation and interpolation. In lay terms, an NLP model is underneath nothing but a curve fitting mechanism, where you are trying to fit a function on to the training data points such that it works well on similar unseen data. If you fit your curve very precisely for the training data, you will achieve excellent accuracy on your training data. But this does not guarantee that your performance on test data will be similar.

Supervised learning is based on the assumption that the unseen test data follows the same distribution or comes from the process/mechanism which is used to generate the training data. So the curve you fitted to your training data can be used to predict the values for unseen test data as well. There are two points to ponder here. For simplicity, we will just consider a one dimensional input  $X$ . We are asked to predict a binary variable 1/0 on this input. Given the height of a person, we are asked to predict whether he is a senior citizen or not. Now, assume that the training data you have seen only consists of inputs that range between 0.9 metres (3 feet) and 1.8 metres (6 feet). You have fitted a curve which explains this training data perfectly. Now let us assume that you are seeing an input of 0.6 metres (2 feet), which is outside the range of the training data you have seen. This is a case of extrapolating the curve to new inputs that were not seen during training. You may also get an input of 1.1 metres (3.5 feet), which was not seen during training. But this input lies in the range you have seen during training. Hence, it is a case of interpolation.

Extrapolation is harder than interpolation. The reason is that when a model is asked to predict something well outside the normal range of data it has seen during training, we can't be sure that the equation of the curve we have fitted to the training data would still be valid for this unseen input. A similar argument does apply for interpolation as well. But since the point lies within the input range seen during training, and given the assumption that the underlying process may not change very rapidly, if there are sufficiently close neighbouring points, interpolation is likely to be more accurate.

While these concepts are typically applied in the context of continuous data, language or text brings with it the added complexity of a discrete nature. Consider an image represented as a high dimensional vector. If you change the input in a few dimensions, the modified input may still be indistinguishable to the human eye and would still be a valid image. On the other hand, think of words being represented in a vector. If you represent words as a one-hot encoding, changing a bit can change the entire word. If you represent words using a word embedding such as Word2Vec or Glove, changing the vector input in a few of the dimensions can result in a new vector which may not map to a valid word.

The inability to generalise across different data sets is not a defect of the NLP model, but is due to the properties of the human language itself. There are inherent variations and ambiguity present in natural language which makes it difficult for machines to handle. After all, a single word can mean a hundred different things, while a hundred different words can convey the same meaning.

Given this complex scenario, the current approaches for assessing NLP model readiness for real world deployment are often ad hoc. An NLP model typically gets evaluated on a few specific test data sets, and its real world readiness is determined based on overall metrics such as accuracy or F1-score on these handful of data sets. However, real world utterances can be extremely diverse – they can have different styles, tones, and be multi-lingual, code mixed, transliterated, misspelled, colloquial, or slang/abbreviated based on a user's educational, demographic and socio-cultural background. Even for a simple sentence like "I love this movie," in addition to the standard paraphrases, we can find other variations, as shown below:

|                         |                                              |
|-------------------------|----------------------------------------------|
| Emojis instead of words | Example: I ♥ this movie                      |
| Slang                   | Example: I love this flick                   |
| Code switched           | Example: I love this <b>ULU</b>              |
| Transliterated          | Example: I love this <i>Patam</i>            |
| Abbreviations           | Example: IMO, Gr8 movie                      |
| Sarcasm/irony           | Example: If you want to burn your money, go! |

Given the inadequacy of test data sets to cover diverse real world utterances, NLP model builders also employ manual beta testing to cover input scenarios that may be missing in the test data sets used for performance validation. Such an evaluation approach is not adequate, and often leads to failures on real world deployment, which leads to an iterative cycle of fail → fix → redeploy. This brings us back to the question: What are the methods we have for evaluating NLP applications for real world scenarios? I would like our readers to think about this problem a bit before we discuss it. If you see the variations of the utterance "I love this movie," can you automatically generate these variations? What techniques would you apply for doing this? We will talk more about this in our column next month.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at [sandyasm\\_AT\\_yahoo\\_DOT\\_com](mailto:sandyasm_AT_yahoo_DOT_com). Wishing all our readers happy coding until next month! 

 By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.